MAT 586
Lab 12

Nolan R. H. Gagnon

Ex. 20.9

The figure below shows the solutions to the differential equations of the SIRS model with $\alpha = 1.4, \gamma = 0.6, \xi = 0.7$, from $t = 0$ to $t = 25$.

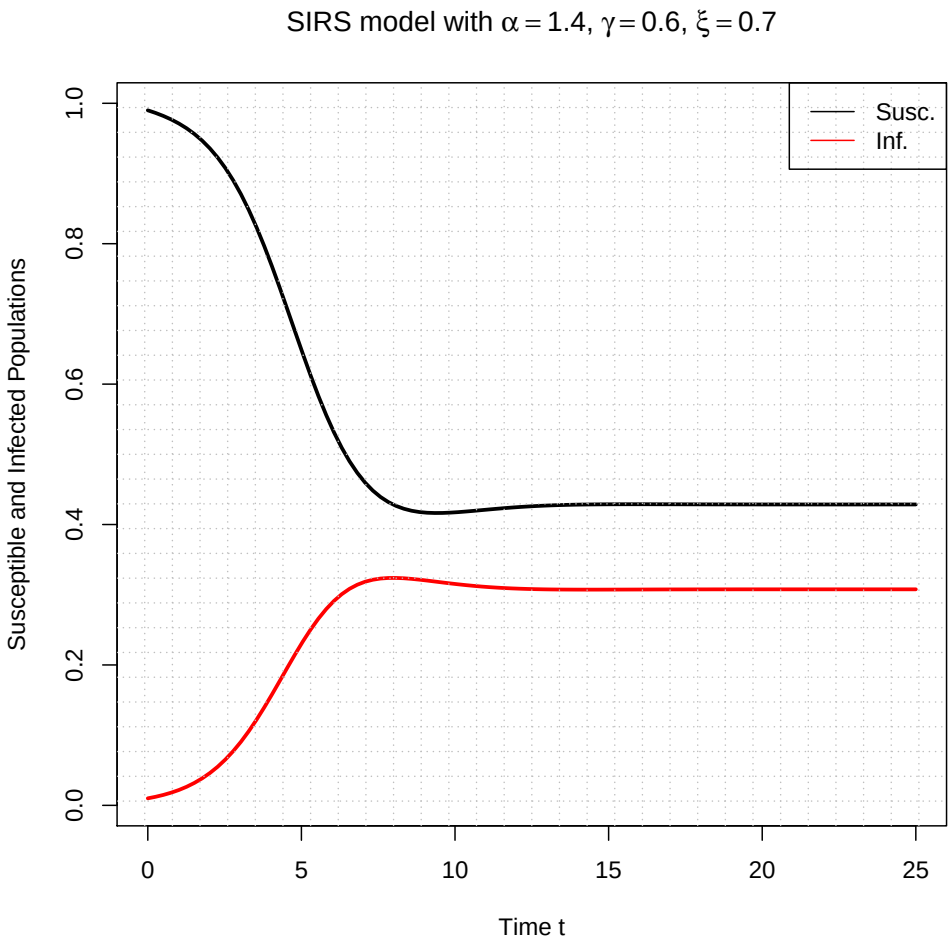

Figure 1: Solutions to SIRS model

The screen-shot on the next page shows an R session in which the population values at $t = 25$ are calculated. It also shows the theoretical equilibrium values and the percentage errors between the final population values and the equilibrium values.

```
> source("exercise_ch20_no9.R")
Warning message:
package 'deSolve' was built under R version 3.4.1
> tmp <- solveSIRS()
> tmp
[[1]]
[1] 0.4285714 0.3076923

[[2]]
[1] 0.4285690 0.3076928

[[3]]
[1] 0.0005615204 0.0001734823

>
```

Figure 2: R Session on Terminal

The first set of values shown in the image above are the theoretical equilibrium values $S^* = 0.4285714$ and $I^* = 0.3076923$. The second set of values contains the final population sizes $S(25) = 0.4285690$ and $I(25) = 0.3076928$. Finally, the third set of values contains the percentage errors between S^* and $S(25)$ and I^* and $I(25)$, respectively. Clearly, the percentage errors are very small, so the predicted endemic equilibrium matches the results of the SIRS simulation.

The R code used for the computations above is provided below.

exercise_ch20_no9.R

```
# provides access to lsoda function for numerically
# solving systems of differential equations
library('deSolve')

# function implementing the SIRS epidemiological model
source('sirs.R')

# function that calculates the equilibrium values for
# the SIRS epidemiological model
source('sirsEquil.R')



## BRIEF: Numerically solves the SIRS epidemiological model and plots
##        the solutions. Also computes equilibrium values for susceptible
##        and infected populations.
##
## PARAM t_start: Where to start finding/plotting SIRS solution.
##
```

```
## PARAM t_stop: Where to stop finding/plotting SIRS solution.
##
## PARAM numSteps: Number of points between t_start and t_stop.
##
## PARAM s_init: Initial condition for susceptible population.
##
## PARAM i_init: Initial condition for infected population.
##
## PARAM alpha: Rate at which individuals encounter each other
##              multiplied by the probability of becoming infected
##              after encountering an infected individual.
##
## PARAM gamma: Recovery rate of infected individual.
##
## PARAM xi: Susceptibility rate of resistant individual.
##
## RETURNS: Equilibrium values for SIRS model, final population values of simulation,
##          and percentage errors between equilibrium and final population values.
solveSIRS <- function(t_start = 0, t_stop = 25, numSteps = 100, s_init = 0.99, i_init = 0.01,
          alpha = 1.4, gamma = 0.6, xi = 0.7)
{
  # numerically solve SIRS model and store results in matrix
  A.m <- lsoda(c(s_init, i_init), seq(t_start, t_stop, length.out = numSteps), sirs, c(alpha, gamma,
      xi))

  # plot solutions to SIRS model
  matplot(A.m[, 1], A.m[, 2:3], type = 'l', lty = 'solid', lwd = 2.5, col = c('black', 'red'),
          xlab = 'Time t', ylab = 'Susceptible and Infected Populations',
          main = bquote(paste("SIRS model with ", alpha == .(alpha), ", ", gamma == .(gamma), ", ", xi
              == .(xi))))
  grid(nx = 30, ny = 30, col = 'gray', lty = 'dotted')
  legend('topright', legend = c('Susc.', 'Inf.'), lty = c('solid', 'solid'), col = c('black', 'red'))

  # Calculate theoretical equilibrium values for
  # susceptible and infected populations
  eq.v <- sirsEquil(alpha = alpha, gamma = gamma, xi = xi)

  # extract the final population values from SIRS simulation run
  final_vals.v <- c(tail(A.m[, 2], 1), tail(A.m[, 3], 1))
```

```
  # calculate percentage error between final population values and
  # theoretical equilibrium values
  eq_pct_err_S <- 100 * abs((eq.v[1] - final_vals.v[1]) / final_vals.v[1])
  eq_pct_err_I <- 100 * abs((eq.v[2] - final_vals.v[2]) / final_vals.v[2])
  eq_pct_err.v <- c(eq_pct_err_S, eq_pct_err_I)

  # return equilibrium values, final population values, and percentage error values to caller
  return(invisible(list(eq.v, final_vals.v, eq_pct_err.v)))
}
```

sirsEquil.R

```
## BRIEF: Computes the equilibrium values for the SIRS epidemiological model.
##
## PARAM alpha: Rate at which individuals encounter each other
##              multiplied by the probability of becoming infected
##              after encountering an infected individual.
##
## PARAM gamma: Recovery rate of infected individual.
##
## PARAM xi: Susceptibility rate of resistant individual.
##
## RETURNS: Equilibrium values for SIRS model in a vector.
sirsEquil <- function(alpha = 1.4, gamma = 0.6, xi = 0.7)
{
  # compute equilibrium value for susceptible population
  s_eq <- gamma / alpha

  # compute equilibrium value for infected population
  i_eq <- (xi / (gamma + xi)) * (1 - gamma / alpha)

  # put both equilibrium values into a vector
  eq.v <- c(s_eq, i_eq)

  # return vector of equilibrium values to the caller
  return(invisible(eq.v))
}
```

sirs.R

```
## BRIEF: Implements the system of differential equations for
##         the SIRS model so they can be solved by lsoda.
##
## PARAM t: Independent variable t (usually time).
##
## PARAM x.v: Values we want to solve differential equations for.
##
## PARAM params.v: Vector of parameters used in differential equations.
##
## RETURNS: Vector of derivatives wrapped in a list.
sirs <- function(t, x.v, params.v)
{
   # extract parameters
   alpha <- params.v[1]
   gamma <- params.v[2]
   xi <- params.v[3]

   # susceptible population
   S <- x.v[1]

   # infected population
   I <- x.v[2]

   # system of differential equations for SIRS model
   dSdt <- -alpha * S * I + xi * (1 - S - I)
   dIdt <- alpha * S * I - gamma * I

   # return diff. eqs. wrapped in list, as required by lsoda
   return(list(c(dSdt, dIdt)))
}
```

■

Ex. 20.10

The figure below shows the solutions to the differential equations of the lattice spatial population model model with $\phi = 2.1$, from $t = 0$ to $t = 20$.

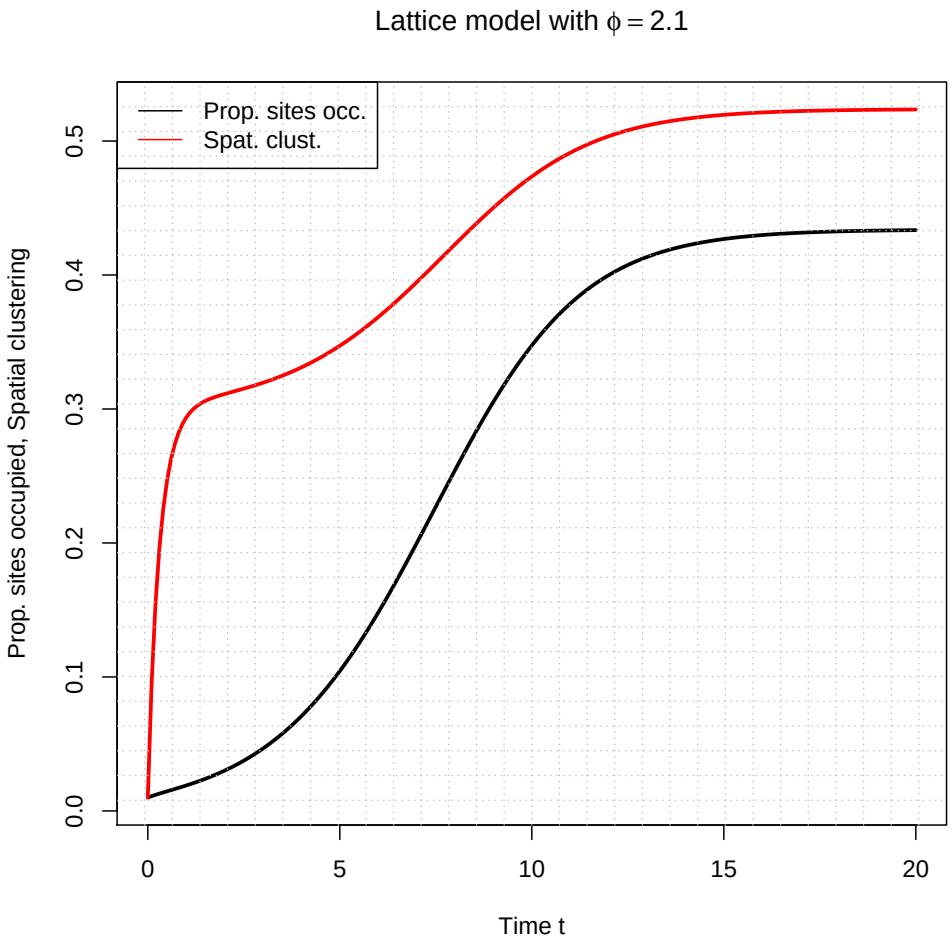

Figure 3: Solutions to Lattice Spatial Population Model

As expected, the spatial clustering (red line) increases rapidly at the start of the simulation, while the population density (black line) increases much more slowly. Eventually the spatial clustering levels off and both spatial clustering and population density approach equilibrium a little after $t = 10$.

In the screen-shot on the following page, an R session is shown in which the final values for population density and spatial clustering (respectively) are computed. The screen-shot indicates that the equilibrium value for population density is approximately 0.4334975 and the equilibrium value for spatial clustering is approximately 0.5235296.

```
> source("exercise_ch20_no10.R")
Warning message:
package 'deSolve' was built under R version 3.4.1
> tmp <- solveLattice()
> tmp
[1] 0.4334975 0.5235296
>
```

Figure 4: R Session on Terminal

The R code used for the computations above is provided below.

exercise_ch20_no10.R

```
# provides access to lsoda function for numerically
# solving systems of differential equations
library('deSolve')

# function implementing the lattice spatial population model
# from Exercise 20.10
source('lattice.R')



## BRIEF: Numerically solves the lattice spatial population model
##        and plots solutions.
##
## PARAM t_start: Where to start finding/plotting lattice solution.
##
## PARAM t_stop: Where to stop finding/plotting lattice solution.
##
## PARAM step_size: Time step size.
##
## PARAM x_init: Initial condition for proportion of sites occupied.
##
## PARAM z_init: Initial condition for spatial clustering.
##
## PARAM phi: Rate at which offspring are produced at occupied site on lattice.
##
## RETURNS: Final values for x and z.
solveLattice <- function(t_start = 0, t_stop = 20, step_size = 0.1, x_init = 0.01, z_init = 0.01, phi
    = 2.1)
{
   # numerically solve lattice model and store results in matrix
```

```
  A.m <- lsoda(c(x_init, z_init), seq(t_start, t_stop, step_size), lattice, c(phi))

  # plot solutions to lattice model
  matplot(A.m[, 1], A.m[, 2:3], type = 'l', lty = 'solid', lwd = 2.5, col = c('black', 'red'),
          xlab = 'Time t', ylab = 'Prop. sites occupied, Spatial clustering',
          main = bquote(paste("Lattice model with ", phi == .(phi))))
  grid(nx = 30, ny = 30, col = 'gray', lty = 'dotted')
  legend('topleft', legend = c('Prop. sites occ.', 'Spat. clust.'), lty = c('solid', 'solid'), col =
      c('black', 'red'))

  # extract the final population values from lattice simulation run
  final_vals.v <- c(tail(A.m[, 2], 1), tail(A.m[, 3], 1))

  # return final x and z values
  return(invisible(final_vals.v))
}
```

lattice.R

```
## BRIEF: Implements the system of differential equations
##        that describe the lattice spatial population model
##        from Exercise 20.10.
##
## PARAM t: Independent variable t (usually time).
##
## PARAM x.v: Values we want to solve differential equations for.
##
## PARAM params.v: Vector of parameters used in differential equations.
##
## RETURNS: Vector of derivatives wrapped in a list
lattice <- function(t, y.v, params.v)
{
  # extract parameters
  phi <- params.v[1]

  # proportion of sites occupied
  x <- y.v[1]

  # probability randomly-chosen neighbor of
```

```
  # randomly-chosen occupied site is occupied
  z <- y.v[2]

  # system of differential equations for lattice model
  dxdt <- x * (1 - z) * phi - x
  dzdt <- (1 - z) * (phi / 2 + (3 * phi * x * (1 - z)) / (2 * (1 - x))) - z * (phi + 1) + phi * z ^ 2

  # return diff. eqs. wrapped in list, as required by lsoda
  return(list(c(dxdt, dzdt)))
}
```

■